

Extra 3 · Normalization and Schema Design

Why this matters in interviews. "What is normalization? Explain 1NF, 2NF, 3NF and BCNF" is asked in almost every database round, and design rounds ask "design the tables for a shopping app / library / hospital". This chapter walks one messy table through every normal form, **executes the decomposition** so you can see nothing is lost, and then covers ER modelling, keys, denormalization and star schemas.

In this chapter: the problem (redundancy and anomalies) · functional dependencies · finding keys with attribute closure · 1NF → 2NF → 3NF → BCNF (worked example) · 4NF and 5NF in brief · denormalization · ER modelling and relationship types · keys (natural vs surrogate) · a design checklist · OLTP vs OLAP.

E3.1 The problem: one big table

A beginner's first design puts everything in one table. Here is an order system as a single table `orders_raw` — one row per product on an order:

orders_raw

order_id	customer_id	customer_name	customer_city	product_id	product_name	unit_price	qty
1	101	Asha	Chennai	11	Pen	10	5
1	101	Asha	Chennai	12	Book	50	1
2	102	Bala	Mumbai	11	Pen	10	20
3	101	Asha	Chennai	13	Bag	400	1
3	101	Asha	Chennai	11	Pen	10	2
4	103	Chitra	Pune	12	Book	50	3

The primary key is (**order_id, product_id**). It works, but look at what is repeated:

- Asha's name and city appear on **four** rows.
- The price of a Pen appears on **three** rows.

Repetition (**redundancy**) causes three kinds of trouble, called **anomalies**:

Anomaly	What goes wrong	In this table
Update	change a fact in one row, forget the others → inconsistent data	Pen's price changes to 12 but two rows still say 10
Insert	can't record a fact without an unrelated fact	can't add a new product "Marker" until someone orders it (there is no <code>order_id</code> to put in the key)
Delete	deleting one fact silently destroys another	delete Chitra's only order and you also lose the fact that Chitra lives in Pune

You can see the repetition directly:

```
SELECT product_id, product_name, unit_price, COUNT(*) AS rows_storing_this_price
FROM orders_raw
GROUP BY product_id, product_name, unit_price;
```

Result

product_id	product_name	unit_price	rows_storing_this_price
11	Pen	10	3
12	Book	50	2
13	Bag	400	1

Normalization is the process of **splitting a table into smaller, well-structured tables** so that each fact is stored **once**, and the pieces can be joined back together to rebuild the original.

E3.2 Functional dependencies (the tool behind every normal form)

A **functional dependency** $X \rightarrow Y$ means: *if two rows have the same X they must have the same Y*. X is the **determinant**. "X determines Y."

In the order system:

Dependency	In words
$\text{customer_id} \rightarrow \text{customer_name, customer_city}$	a customer id fixes the name and city
$\text{product_id} \rightarrow \text{product_name, unit_price}$	a product id fixes its name and price
$\text{order_id} \rightarrow \text{customer_id}$	an order belongs to exactly one customer
$(\text{order_id, product_id}) \rightarrow \text{qty}$	the quantity depends on the order and the product together

A few vocabulary words:

Term	Meaning
Superkey	any set of columns that determines every column
Candidate key	a minimal superkey (remove any column and it stops working)
Primary key	the candidate key you choose
Prime attribute	a column that is part of <i>some</i> candidate key; the others are non-prime
Composite key	a key made of more than one column

Finding keys with attribute closure

The **closure** X^+ is the set of all columns you can reach from X by repeatedly applying the dependencies. If X^+ is every column, X is a superkey.

Worked example. R(A, B, C, D, E) with $A \rightarrow B$, $B \rightarrow C$, $C D \rightarrow E$. Is $\{A, D\}$ a key?

1. Start: $\{A, D\}$
2. $A \rightarrow B \Rightarrow \{A, B, D\}$

3. $B \rightarrow C \Rightarrow \{A, B, C, D\}$
4. $C D \rightarrow E \Rightarrow \{A, B, C, D, E\}$ — **everything**, so $\{A, D\}$ is a superkey.
5. Is it minimal? $\{A\}^+ = \{A, B, C\}$ and $\{D\}^+ = \{D\}$ — neither is everything, so **yes**, $\{A, D\}$ is a candidate key.

Tip. Any column that never appears on the *right side* of a dependency must be in every key. Start there.

Rules of inference (Armstrong's axioms), useful to name in interviews: **reflexivity** (if $Y \subseteq X$ then $X \rightarrow Y$), **augmentation** ($X \rightarrow Y \Rightarrow XZ \rightarrow YZ$), **transitivity** ($X \rightarrow Y$ and $Y \rightarrow Z \Rightarrow X \rightarrow Z$). Derived: union, decomposition, pseudo-transitivity.

E3.3 First Normal Form (1NF)

Rule: every column holds a **single, atomic value**; no repeating groups, no lists in one cell; every row is identifiable (a key).

Not 1NF — a list in a column:

order_id	customer	products
1	Asha	Pen, Book
2	Bala	Pen

Problems: you can't filter `WHERE product = 'Book'` properly, can't count products, can't enforce a foreign key.

Also not 1NF — repeating columns:

order_id	product1	product2	product3

Fix: **one row per value**, the layout `orders_raw` already uses (one row per order line). Our starting table is in 1NF.

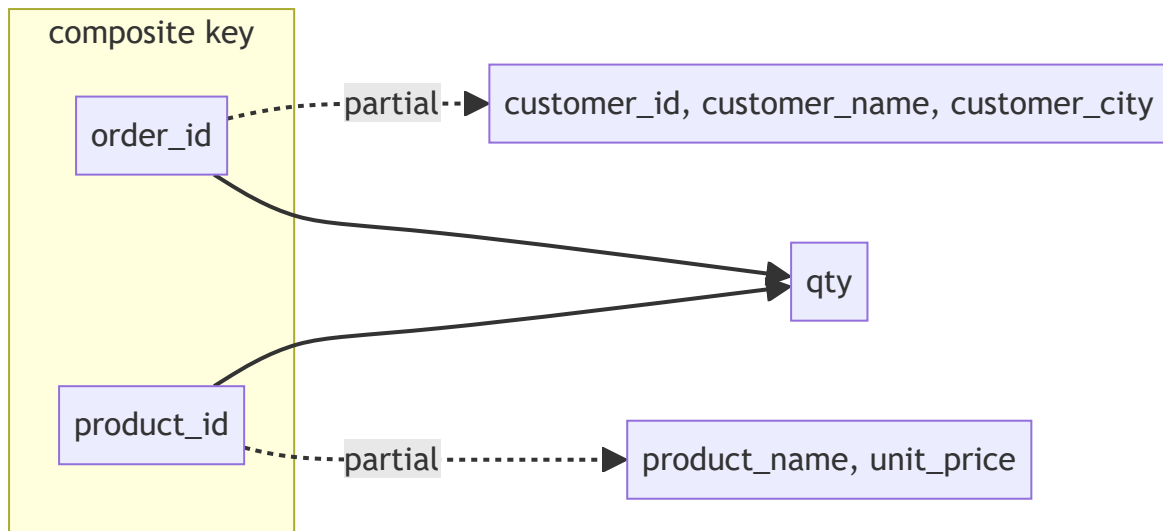
E3.4 Second Normal Form (2NF)

Rule: the table is in 1NF **and** no non-key column depends on **only part** of a composite key (**no partial dependency**).

(A table whose key is a single column is automatically in 2NF.)

In `orders_raw` the key is $(order_id, product_id)$, but:

- `product_name` and `unit_price` depend only on `product_id` ← part of the key
- `customer_id`, `customer_name` and `customer_city` depend only on `order_id` ← part of the key
- only `qty` depends on the **whole** key



Fix: move every partially dependent group into its own table, keyed by the part it depends on.

Table	Key	Columns
products	product_id	product_name , unit_price
orders	order_id	customer_id , customer_name , customer_city
order_items	(order_id, product_id)	qty

Let's actually do it and check that we can rebuild the original:

```

CREATE TABLE products AS SELECT DISTINCT product_id, product_name, unit_price FROM orders_raw;
CREATE TABLE orders_2nf AS SELECT DISTINCT order_id, customer_id, customer_name, customer_city FROM orders_raw;
CREATE TABLE order_items AS SELECT order_id, product_id, qty FROM orders_raw;
SELECT * FROM products ORDER BY product_id;
  
```

Result

product_id	product_name	unit_price
11	Pen	10
12	Book	50
13	Bag	400

Table after

order_id	customer_id	customer_name	customer_city
1	101	Asha	Chennai
2	102	Bala	Mumbai
3	101	Asha	Chennai
4	103	Chitra	Pune

products now stores each price **once** (the third table, orders_2nf , is shown below the result).

E3.5 Third Normal Form (3NF)

Rule: the table is in 2NF **and** no non-key column depends on **another non-key column** (no transitive dependency).

The famous one-liner: "Every non-key attribute must provide a fact about the key, the whole key, and nothing but the key — so help me Codd."

`orders_2nf` still has a problem:

```
order_id → customer_id → customer_name, customer_city
```

`customer_name` doesn't depend on the order directly; it depends on `customer_id`, which depends on the order. That is a **transitive dependency**: Asha's details are repeated for every one of her orders.



Fix: split the customer out.

```
CREATE TABLE customers AS SELECT DISTINCT customer_id, customer_name, customer_city FROM orders_raw;
CREATE TABLE orders_3nf AS SELECT DISTINCT order_id, customer_id FROM orders_raw;
SELECT * FROM customers ORDER BY customer_id;
```

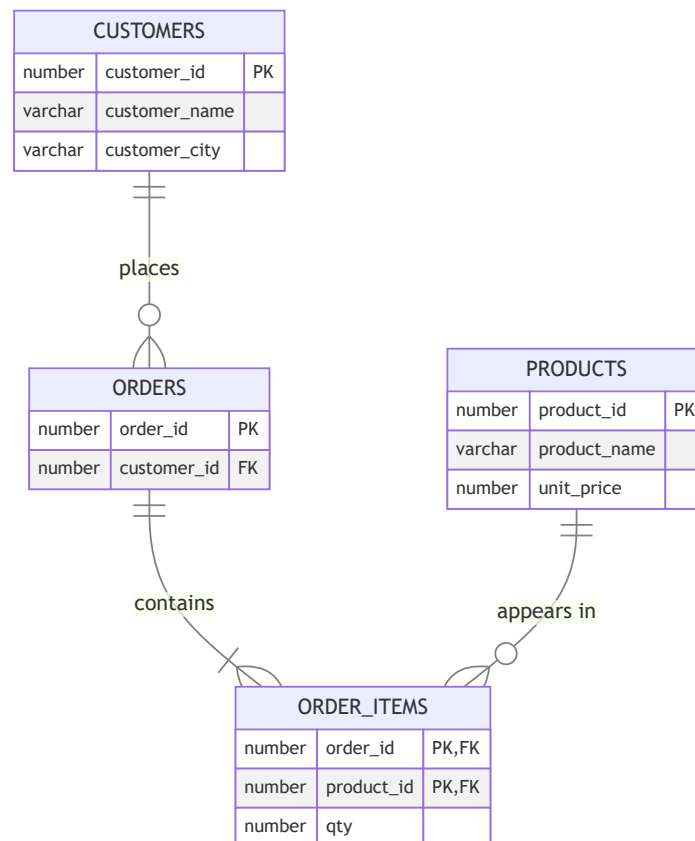
Result

customer_id	customer_name	customer_city
101	Asha	Chennai
102	Bala	Mumbai
103	Chitra	Pune

Table after

order_id	customer_id
1	101
2	102
3	101
4	103

The final design (3NF):



Proof that nothing was lost (a *lossless decomposition*)

Join the four tables back together — we must get exactly the original six rows. `EXCEPT` (Oracle: `MINUS`) shows any difference:

```

CREATE TABLE customers AS SELECT DISTINCT customer_id, customer_name, customer_city FROM
orders_raw;
CREATE TABLE orders_3nf AS SELECT DISTINCT order_id, customer_id FROM orders_raw;
CREATE TABLE products AS SELECT DISTINCT product_id, product_name, unit_price FROM orders_raw;
CREATE TABLE order_items AS SELECT order_id, product_id, qty FROM orders_raw;
SELECT o.order_id, c.customer_id, c.customer_name, c.customer_city,
       p.product_id, p.product_name, p.unit_price, i.qty
FROM   order_items i
JOIN   orders_3nf o ON o.order_id = i.order_id
JOIN   customers c ON c.customer_id = o.customer_id
JOIN   products p ON p.product_id = i.product_id
EXCEPT
SELECT order_id, customer_id, customer_name, customer_city, product_id, product_name, unit_price,
qty
FROM   orders_raw;
  
```

Result

order_id	customer_id	customer_name	customer_city	product_id	product_name	unit_price	qty
(no rows)							

(no rows) — the rebuilt data has nothing that the original lacked. Together with the reverse check (the original has nothing the rebuild lacks) this proves the join reconstructs `orders_raw` exactly.

The payoff — the anomalies are gone

Anomaly	Now
Update	change a price in one row of <code>products</code>
Insert	add a product with no orders — just insert into <code>products</code>
Delete	deleting an order leaves <code>customers</code> and <code>products</code> untouched

How it looks in Oracle DDL

```
CREATE TABLE customers (  
  customer_id  NUMBER          PRIMARY KEY,  
  customer_name VARCHAR2(50) NOT NULL,  
  customer_city VARCHAR2(50)  
);  
  
CREATE TABLE products (  
  product_id   NUMBER          PRIMARY KEY,  
  product_name VARCHAR2(50) NOT NULL,  
  unit_price   NUMBER(10,2) CHECK (unit_price >= 0)  
);  
  
CREATE TABLE orders (  
  order_id      NUMBER          PRIMARY KEY,  
  customer_id   NUMBER          NOT NULL REFERENCES customers (customer_id)  
);  
  
CREATE TABLE order_items (  
  order_id      NUMBER          REFERENCES orders (order_id),  
  product_id    NUMBER          REFERENCES products (product_id),  
  qty           NUMBER          CHECK (qty > 0),  
  PRIMARY KEY (order_id, product_id)  
);
```

A design point worth stating aloud. `unit_price` lives in `products`, but an old order should keep the price it was *sold at* even if the product's price changes later. Real systems therefore copy the price into `order_items` (as `sold_price`). That is **deliberate, documented duplication** — a different fact ("price at the time of sale"), not an anomaly.

E3.6 Boyce–Codd Normal Form (BCNF)

Rule: for **every** functional dependency $X \rightarrow Y$, X must be a **superkey**. ("Every determinant is a candidate key.")

BCNF is a slightly stricter 3NF. They differ only in a rare case: a table with **overlapping candidate keys** where a **non-key column determines part of a key**.

Classic example. A student takes a course and is taught by one instructor; each instructor teaches only **one** course (but a course can have several instructors).

student	course	instructor
Asha	DBMS	Rao
Bala	DBMS	Rao
Asha	OS	Iyer
Chitra	DBMS	Menon

Dependencies: $(\text{student}, \text{course}) \rightarrow \text{instructor}$ and $\text{instructor} \rightarrow \text{course}$.

- Candidate keys: $(\text{student}, \text{course})$ and $(\text{student}, \text{instructor})$. So **every column is prime**, hence the table **is in 3NF** (3NF forgives a determinant whose target is a prime attribute).
- But $\text{instructor} \rightarrow \text{course}$ has a determinant (instructor) that is **not a superkey** $\Rightarrow$ **not BCNF**. Symptom: change Rao's course and you must update every row of Rao's students.

Fix (BCNF): split into $\text{teaches}(\text{instructor}, \text{course})$ and $\text{enrolled}(\text{student}, \text{instructor})$.

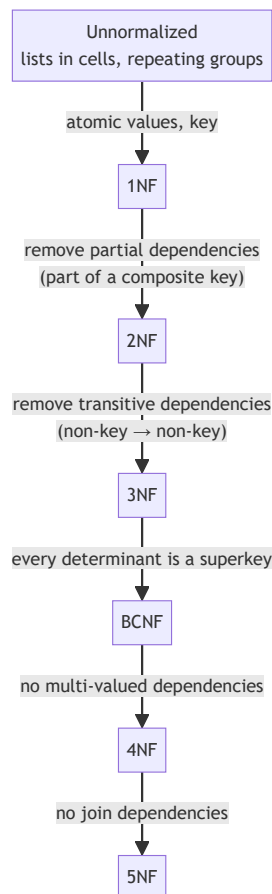
Trade-off. BCNF decomposition can lose the ability to enforce some dependency (here, $(\text{student}, \text{course}) \rightarrow \text{instructor}$) with a simple key/unique constraint. That is why 3NF is often the practical stopping point; **3NF can always be reached with lossless and dependency-preserving decomposition, BCNF cannot always.**

E3.7 Fourth and Fifth Normal Forms (brief)

Form	Rule	Typical trigger
4NF	no non-trivial multi-valued dependency except on a superkey	one table storing two <i>independent</i> lists — e.g. $(\text{employee}, \text{skill}, \text{language})$ with rows for every skill $\times$ language combination. Split into employee_skill and employee_language
5NF	no join dependency that isn't implied by the keys	a fact that only makes sense as a three-way relationship, which can be rebuilt from three two-way tables only if a rule holds. Rare in practice

You rarely need more than 3NF/BCNF in practice, but be ready to explain 4NF with the skill $\times$ language example.

E3.8 The normal forms on one page



Normal form	The violation it removes	The cure
1NF	non-atomic values / repeating groups	one value per cell, one row per value
2NF	partial dependency (part of a composite key → column)	split into a table per part of the key
3NF	transitive dependency (non-key → non-key)	move the dependent group to its own table
BCNF	a determinant that isn't a superkey	split on that determinant
4NF	independent multi-valued facts in one table	one table per independent fact
5NF	join dependency not implied by keys	split into the minimal pieces

E3.9 Denormalization

Denormalization is *deliberately* going back — storing redundant data — to make reads faster.

When it is reasonable	Examples
Reporting / analytics that always join the same 6 tables	a wide summary table refreshed nightly
Read-heavy, rarely-changing data	copy <code>customer_name</code> into an order for a receipt
Computed values that are expensive	store <code>order_total</code> on the order
Historical accuracy	the sold price, the shipping address <i>at the time</i>

Costs: more storage, updates must maintain every copy (triggers, application code or a materialised view), and anomalies return. The rule: **normalize first; denormalize only for a measured performance problem, and document it.**

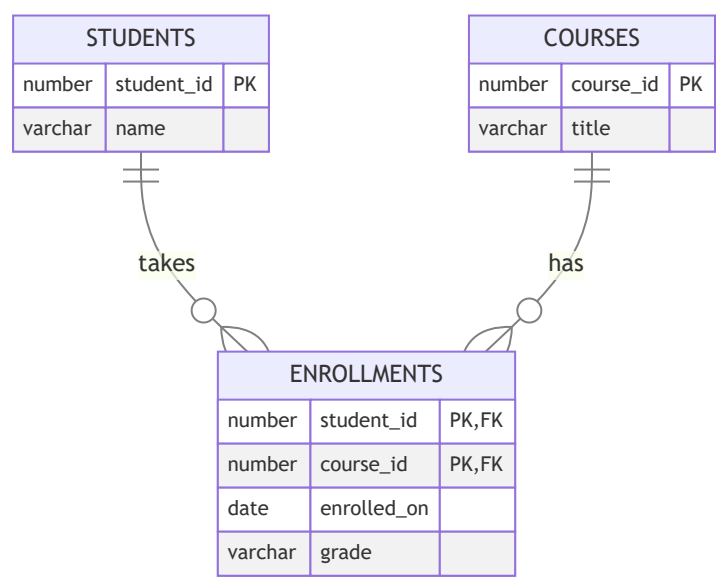
E3.10 ER modelling in 5 minutes

An **Entity–Relationship** model is the design you draw *before* the tables.

Concept	Meaning	Becomes
Entity	a thing you store facts about (Customer, Product)	a table
Attribute	a fact about it (name, city)	a column
Relationship	how entities connect (Customer <i>places</i> Order)	a foreign key (or a junction table)
Cardinality	how many on each side: 1:1, 1:N, M:N	where the foreign key goes
Weak entity	can't be identified without its owner (an order <i>line</i>)	table with a composite key including the owner's key

Mapping relationships to tables

Relationship	Example	Tables
One-to-many (1:N)	one customer, many orders	put the FK on the "many" side (<code>orders.customer_id</code>)
One-to-one (1:1)	a person and their passport	FK with a UNIQUE constraint, or share the primary key
Many-to-many (M:N)	students ↔ courses	a junction (bridge) table holding both foreign keys; the pair is its primary key



The junction table is also where attributes of the *relationship itself* (`grade` , `enrolled_on`) live — they belong to neither student nor course.

A relational table cannot represent M:N directly — always convert it to two 1:N relationships around a junction table.

Reading crow's-foot notation

`||` exactly one · `o|` zero or one · `|{` one or many · `o{` zero or many. So `CUSTOMERS ||--o{ ORDERS` reads "a customer places zero or many orders; an order belongs to exactly one customer."

E3.11 Keys — natural vs surrogate

	Natural key	Surrogate key
What	a real-world value (email, ISBN, PAN)	a meaningless generated number (identity/sequence)
For	small stable codes; lookup tables	most entity tables
Pros	meaningful, no extra column	never changes, small, fast joins, no privacy leak
Cons	can change (emails do), can be long, may be composite	carries no meaning, so add a <code>UNIQUE</code> on the natural key too

Common practice: **surrogate primary key** + `UNIQUE` constraint on the natural key.

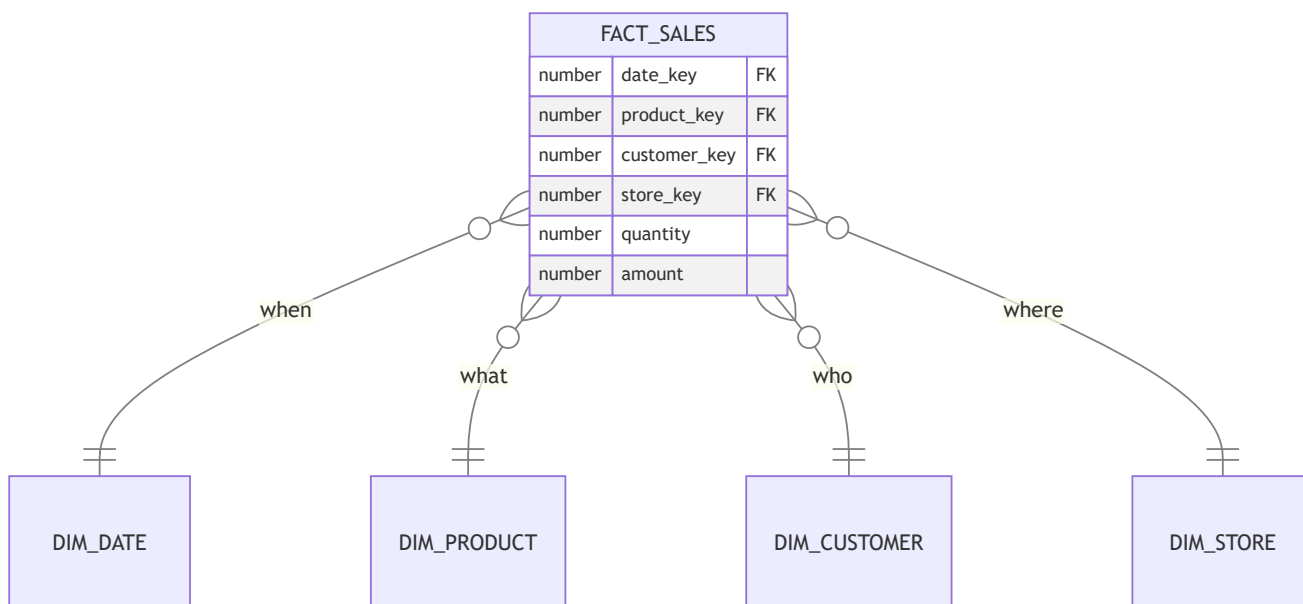
E3.12 A design checklist for "design the schema" questions

1. **List the entities** (nouns) and their attributes.
2. **Find the relationships** (verbs) and decide 1:1 / 1:N / M:N; add junction tables for M:N.
3. Give every table a **primary key**; add **foreign keys** with the right `ON DELETE` (`CASCADE` for owned children like order lines, `restrict`/none for shared parents).
4. **Normalize to 3NF**: one fact in one place.
5. Add constraints: `NOT NULL` , `UNIQUE` , `CHECK` , `DEFAULT` .
6. Choose data types carefully (money → `NUMBER(p,s)` , never floating point).
7. **Index** foreign keys and columns you filter/join on.
8. Consider **history** (audit tables, soft deletes, effective dates) and **denormalize only if measured**.

E3.13 OLTP vs OLAP — and the star schema

	OLTP (running the business)	OLAP (analysing it)
Workload	many small reads/writes	few large read-only aggregations
Design	normalized (3NF)	denormalized star/snowflake
Examples	banking, orders, ticketing	dashboards, data warehouse

A **star schema** has one central **fact table** (measurable events: sales) surrounded by **dimension tables** (who, what, when, where) — deliberately denormalized so reports need few joins.



A **snowflake schema** further normalizes the dimensions (product → category → department).

Interview corner

Q. What is normalization and why do it? Organising tables to reduce redundancy and avoid update, insert and delete anomalies by storing each fact once and linking with keys.

Q. Explain 1NF, 2NF and 3NF in one sentence each. 1NF: atomic values, no repeating groups. 2NF: 1NF and no column depends on *part* of a composite key. 3NF: 2NF and no non-key column depends on another non-key column — "the key, the whole key, and nothing but the key."

Q. 3NF vs BCNF? BCNF requires every determinant to be a superkey. They differ only when a table has overlapping candidate keys and a non-key column determines part of a key. 3NF always preserves dependencies; BCNF may not.

Q. What is a functional dependency? Give an example. $X \rightarrow Y$: each X value determines exactly one Y value — `emp_id → emp_name`.

Q. What is a partial dependency? A transitive dependency? Partial: a non-key column depends on only part of a composite key (breaks 2NF). Transitive: non-key → non-key (breaks 3NF).

Q. What are the anomalies? Update (inconsistent copies), insert (can't record a fact without another), delete (removing one fact loses another).

Q. What is denormalization and when is it acceptable? Adding redundancy on purpose to speed reads — for reporting, read-heavy data, expensive computed values, historical snapshots — after normalizing and measuring.

Q. How do you model a many-to-many relationship? A junction table with foreign keys to both sides; the pair is the primary key; relationship attributes go there.

Q. Natural vs surrogate key? Natural has business meaning but can change; surrogate is a generated number that never changes. Usually surrogate PK plus a unique constraint on the natural key.

Q. Is a lossless decomposition enough? It must also ideally be **dependency-preserving**. Lossless means joining the pieces returns exactly the original (no lost or spurious rows).

Quick recap

- **Anomalies** (update / insert / delete) come from redundancy; **normalization** removes it by splitting tables.
- FD $X \rightarrow Y$; key = minimal set whose **closure** is every column.
- **1NF** atomic · **2NF** no partial dependency · **3NF** no transitive dependency · **BCNF** every determinant is a superkey · **4NF** no independent multi-valued facts.
- Decomposition must be **lossless** (join gives the original back) and ideally dependency-preserving.
- Model: 1:N → FK on many side; M:N → junction table; 1:1 → unique FK.
- Denormalize only deliberately; OLTP = normalized, OLAP = star schema.